RELATÓRIO DE TRABALHO PRÁTICO 1:
O PROBLEMA DOS FILÓSOFOS JANTANDO EM GO

Fundamentos de Processamento Paralelo e Distribuído (FPPD)

1. Identificação do Grupo e Ambiente de Execução

• **Integrantes:** Guilherme Couto de Castro e Juliano Machado da Silva
• **Repositório Git:** https://github.com/GuilhermeCoutodeCastro/T1_FPPD_Jantar_dos_filosofos
• **Linguagem e Versão:** Go (Golang) v1.26.6
• **Sistema Operacional:** Windows 11 Home Single Language (64 bits)
• **Processador e Cores:** Intel(R) Core(TM) i5-11400H @ 2.70GHz (6 núcleos físicos, 12 processadores lógicos)
• **Comandos de Execução:** go run v0_sequencial.go | go run v1_deadlock.go | go run v2_sincronizacao_excessiva.go | go run v3_aprimorada.go

2. Descrição das Soluções e Mecanismos de Sincronização

O trabalho analisa quatro estratégias para o Problema dos Filósofos Jantando:

1. V0 - Execução Sequencial: Execução direta em malha sequencial sem concorrência. Cada filósofo pensa e come ordenadamente um por vez.

2. V1 - Concorrente Ingênua (Deadlock): Cada filósofo é representado por uma goroutine e tenta alocar o garfo esquerdo e depois o direito. Quando todos seguram o garfo esquerdo ao mesmo tempo, forma-se uma espera circular e o programa trava (deadlock).

3. V2 - Sincronização Excessiva (Árbitro Global): Utiliza um Mutex de árbitro global que serializa a fase de aquisição dos garfos: apenas um filósofo por vez pode tentar pegar os dois garfos. Isso evita o deadlock, mas cria um gargalo, pois enquanto esse filósofo aguarda um garfo ocupado, todos os demais que querem comer ficam bloqueados no árbitro.

4. V3 - Concorrente Aprimorada (Hierarquia de Recursos): Aplica a ordenação estrita de recursos, obrigando cada filósofo a tentar adquirir primeiro o garfo de menor ID global, eliminando a espera circular.

Análise Comparativa: Com base na média de 5 execuções e tomando a V0 (656.94 ms) como baseline, a V2 (234.03 ms) e a V3 (202.75 ms) reduziram o tempo total para cerca de um terço, com até 2 filósofos comendo simultaneamente. A V1 travou em deadlock antes da primeira refeição. Entre as soluções corretas, a V3 foi superior em todas as métricas: menor tempo total, menor espera média (3.21 ms contra 4.88 ms), menor pico de espera (12.47 ms contra 23.84 ms) e resultados mais estáveis entre execuções. Assim, mais sincronização garantiu a corretude, mas não trouxe melhor desempenho.

3. Tabela de Resultados Experimentais e Evidências Visuais

Métrica / Solução	V0 (Sequencial)	V1 (Deadlock)	V2 (Árbitro)	V3 (Hierarquia)
Mecanismo Principal	Sequencial puro	sync.Mutex por garfo	sync.Mutex Árbitro + Garfos	Ordenação de Recursos
Tempo Total Médio (ms)	656.94 ms	FALHA (deadlock)	234.03 ms	202.75 ms
Espera Média / Ciclo (ms)	0.00 ms	Indefinida	4.88 ms	3.21 ms
Maior Espera Observada (ms)	0.00 ms	Indefinida	23.84 ms	12.47 ms
Máx. Comendo Simultâneos	1	0	2	2
Ciclos Concluídos	20 / 20	0 / 20	20 / 20	20 / 20

Tabela 1: Valores médios de 5 execuções por versão. A maior espera corresponde ao maior valor observado entre as 5 execuções. A V1 foi executada até a ocorrência do deadlock.

Métrica (ms)	Exec. 1	Exec. 2	Exec. 3	Exec. 4	Exec. 5	Média	Desvio Padrão
V0 – Tempo Total	661.07	615.56	634.00	747.15	626.91	656.94	53.14
V2 – Tempo Total	213.11	250.91	211.60	217.44	277.08	234.03	28.96
V2 – Espera Média	3.87	4.94	4.19	4.33	7.10	4.88	1.30
V2 – Maior Espera	7.69	23.84	7.36	9.51	13.46	12.37	6.85
V3 – Tempo Total	202.95	196.48	205.23	216.32	192.77	202.75	9.07
V3 – Espera Média	3.47	3.19	3.28	3.12	3.00	3.21	0.18
V3 – Maior Espera	12.47	10.28	10.09	10.30	9.79	10.59	1.07

Tabela 2: Resultados individuais das 5 execuções de cada versão (valores em ms).

Gráficos Comparativos de Desempenho

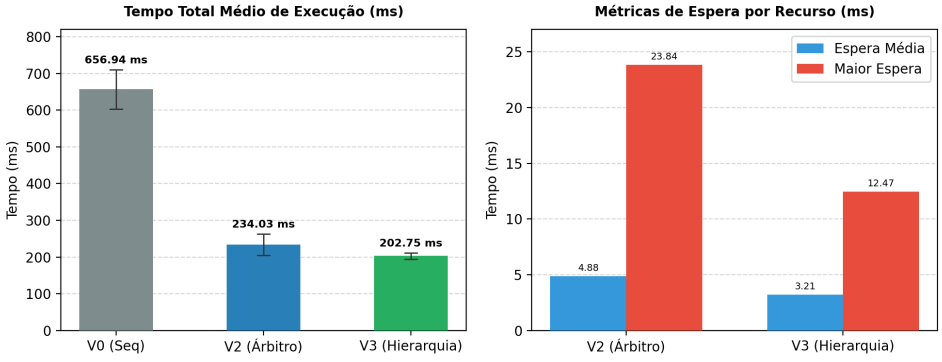


Figura 1: Médias de 5 execuções. Tempo total (barras de erro = desvio padrão), espera média e maior espera observada.

Evidências Visuais das Execuções (pasta prints: V0, V1, V2 e V3)

Execução 1 (v0_0.png)

```
=== V0 Sequencial Concluído ===
Tempo Total de Execução: 661.0657ms
Total de Ciclos por Filósofo: 20
Máximo de Filósofos Comendo Simultaneamente: 1
```

Execução 2 (v0_1.png)

```
=== V0 Sequencial Concluído ===
Tempo Total de Execução: 615.5564ms
Total de Ciclos por Filósofo: 20
Máximo de Filósofos Comendo Simultaneamente: 1
```

Execução 3 (v0_2.png)

```
=== V0 Sequencial Concluído ===
Tempo Total de Execução: 633.9974ms
Total de Ciclos por Filósofo: 20
Máximo de Filósofos Comendo Simultaneamente: 1
```

Execução 4 (v0_3.png)

```
=== V0 Sequencial Concluído ===
Tempo Total de Execução: 747.1473ms
Total de Ciclos por Filósofo: 20
Máximo de Filósofos Comendo Simultaneamente: 1
```

Execução 5 (v0_4.png)

```
=== V0 Sequencial Concluído ===
Tempo Total de Execução: 626.9089ms
Total de Ciclos por Filósofo: 20
Máximo de Filósofos Comendo Simultaneamente: 1
```

Figura 2: V0 - Resumo das 5 execuções (v0_0 a v0_4), com tempo total entre 615.56 ms e 747.15 ms.

```
PS C:\Users\Guilherme\OneDrive - PUCRS - BR\Área de Trabalho\Trabalho FPPD> go run .\v1_deadlock.go

 [P0] [P1] [P2] [P3] [P4]
                      T1
                  T1
       T1
 T1
             T1
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [sync.WaitGroup.Wait]:
sync.runtime_SemacquireWaitGroup(0x5388de3a2d0?, 0x0?)
        C:/Program Files/Go/src/runtime/sema.go:114 +0x2e
sync.(*WaitGroup).Wait(0x5388de220a0)
        C:/Program Files/Go/src/sync/waitgroup.go:206 +0x85
main.main()
        C:/Users/Guilherme/OneDrive - PUCRS - BR/Área de Trabalho/Trabalho FPPD/v1_deadlock.go:76 +0x16f

goroutine 7 [sync.Mutex.Lock]:
internal/sync.runtime_SemacquireMutex(0x4?, 0x70?, 0x1?)
        C:/Program Files/Go/src/runtime/sema.go:95 +0x25
internal/sync.(*Mutex).lockSlow(0x5388de220b8)
        C:/Program Files/Go/src/internal/sync/mutex.go:149 +0x15d
internal/sync.(*Mutex).Lock(...)
```

Figura 3: v1.png - Evidência do erro fatal e travamento de goroutines no runtime do Go (V1).

Execução 1 (v2_0.png)

```
=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 213.1149ms
Tempo Médio de Espera por Ciclo: 3.866316ms
Maior Tempo de Espera Observado: 7.6922ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 2 (v2_1.png)

```
=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 250.9059ms
Tempo Médio de Espera por Ciclo: 4.935937ms
Maior Tempo de Espera Observado: 23.8352ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 3 (v2_2.png)

```
=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 211.5964ms
Tempo Médio de Espera por Ciclo: 4.19336ms
Maior Tempo de Espera Observado: 7.3572ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 4 (v2_3.png)

```
=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 217.4387ms
Tempo Médio de Espera por Ciclo: 4.327547ms
Maior Tempo de Espera Observado: 9.51ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 5 (v2_4.png)

```
=== V2 Sincronização Excessiva Concluída ===
Tempo Total de Execução: 277.0781ms
Tempo Médio de Espera por Ciclo: 7.100278ms
Maior Tempo de Espera Observado: 13.4635ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Figura 4: V2 - Resumo das 5 execuções (v2_0 a v2_4), com tempo total entre 211.60 ms e 277.08 ms.

Execução 1 (v3_0.png)

```
=== V3 Concorrente Aprimorada Concluída ===
Tempo Total de Execução: 202.9545ms
Tempo Médio de Espera por Ciclo: 3.467585ms
Maior Tempo de Espera Observado: 12.4676ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 2 (v3_1.png)

```
=== V3 Concorrente Aprimorada Concluída ===
Tempo Total de Execução: 196.4756ms
Tempo Médio de Espera por Ciclo: 3.185524ms
Maior Tempo de Espera Observado: 10.2772ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 3 (v3_2.png)

```
=== V3 Concorrente Aprimorada Concluída ===
Tempo Total de Execução: 205.2295ms
Tempo Médio de Espera por Ciclo: 3.283452ms
Maior Tempo de Espera Observado: 10.0943ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 4 (v3_3.png)

```
=== V3 Concorrente Aprimorada Concluída ===
Tempo Total de Execução: 216.3223ms
Tempo Médio de Espera por Ciclo: 3.11951ms
Maior Tempo de Espera Observado: 10.3013ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Execução 5 (v3_4.png)

```
=== V3 Concorrente Aprimorada Concluída ===
Tempo Total de Execução: 192.7692ms
Tempo Médio de Espera por Ciclo: 2.995181ms
Maior Tempo de Espera Observado: 9.7877ms
Máximo de Filósofos Comendo Simultaneamente: 2
Total de Ciclos Concluídos por Filósofo: 20
```

Figura 5: V3 - Resumo das 5 execuções (v3_0 a v3_4), com tempo total entre 192.77 ms e 216.32 ms.

4. Conclusão e Respostas às 12 Perguntas

1. Como funciona cada solução?

A V0 executa em uma única thread, chamando cada filósofo em sequência: cada um pensa e come ordenadamente, um por vez. A V1 cria uma goroutine por filósofo; cada uma bloqueia o garfo da esquerda, faz uma pausa de 1 ms e então tenta bloquear o da direita, o que leva todos a segurarem

um garfo e esperarem pelo outro (deadlock). A V2 envolve a aquisição dos dois garfos em um Mutex árbitro global: apenas um filósofo por vez executa a fase de aquisição e libera o árbitro assim que obtém os dois garfos. A V3 impõe uma ordem global de aquisição: cada filósofo pede primeiro o garfo de menor ID e depois o de maior ID (assim, P4 pega o garfo 0 antes do garfo 4), quebrando a simetria entre os filósofos.

2. Quais mecanismos de sincronização são utilizados?

São utilizados Mutexes (sync.Mutex) para garantir a exclusão mútua sobre os garfos e sobre o árbitro central, e WaitGroups (sync.WaitGroup) para coordenar a sincronização e o encerramento das goroutines concorrentes na função main.

3. Como esses mecanismos são implementados na linguagem Go?

Os Mutexes são declarados como sync.Mutex e manipulados com os métodos Lock() para bloqueio e Unlock() para liberação das seções críticas. O WaitGroup utiliza wg.Add(1) ao disparar goroutines, defer wg.Done() para sinalizar o término e wg.Wait() para bloquear o encerramento do programa principal.

4. Por que ocorre deadlock na V1?

Na execução registrada (Figura 3), os 5 filósofos imprimiram T1 e nenhum chegou a comer: cada filósofo Pi detém o garfo i (esquerdo) e espera o garfo (i+1) mod 5 (direito), que está com o vizinho Pi+1. Estão presentes as 4 condições de Coffman: 1) Exclusão Mútua (cada garfo é um sync.Mutex e só pode ter um dono); 2) Posse e Espera (cada filósofo retém o garfo esquerdo e aguarda o direito); 3) Ausência de Preempção (um garfo não pode ser retirado de quem o segura); e 4) Espera Circular (P0 espera por P1, P1 por P2, P2 por P3, P3 por P4 e P4 por P0). O runtime do Go detecta que todas as goroutines estão bloqueadas e encerra o programa com "fatal error: all goroutines are asleep - deadlock!".

5. Como V2 e V3 evitam deadlock?

A V2 evita o deadlock eliminando a Espera Circular de forma indireta: como o árbitro permite que apenas um filósofo por vez esteja na fase de aquisição, os garfos ocupados pertencem a filósofos que já possuem os dois garfos e estão comendo, e que vão liberá-los ao terminar; assim, nunca se forma um ciclo de espera. A Posse e Espera não é eliminada, pois o filósofo que detém o árbitro pode segurar o garfo esquerdo enquanto aguarda o direito. A V3 também elimina a Espera Circular, mas por ordenação de recursos: como todos adquirem os garfos em ordem crescente de ID, P0 e P4 disputam primeiro o garfo 0, e é impossível que cada filósofo segure um garfo e espere pelo próximo em ciclo.

6. Existe possibilidade de starvation? Justifique.

Teoricamente sim. Nenhuma das soluções garante justiça de forma explícita: na V3, um filósofo pode perder repetidamente a disputa pelo primeiro garfo para um vizinho (por exemplo, P0 e P4 disputam o garfo 0), e na V2 a ordem de entrada no árbitro não é garantida. Na prática, o risco é reduzido pelo próprio sync.Mutex do Go, que entra em modo de inanição (starvation mode) quando uma goroutine espera mais de 1 ms e passa a entregar o lock em ordem FIFO. Os dados mostram que não houve starvation: em todas as 5 execuções da V2 e da V3, todos os filósofos concluíram 20/20 ciclos. Ainda assim, a V2 apresentou picos de espera maiores e mais irregulares (maior

espera entre 7.36 ms e 23.84 ms, conforme a execução) do que a V3 (entre 9.79 ms e 12.47 ms), indicando que o árbitro global é a solução mais sujeita a esperas longas.

7. Qual solução apresentou maior tempo total de execução?

A V0 (Sequencial) apresentou o maior tempo total, com média de 656.94 ms nas 5 execuções, pois não explora nenhum paralelismo: os 100 ciclos (5 filósofos × 20 ciclos) de pensar (2 ms) e comer (3 ms) são executados um após o outro, o que dá um mínimo teórico de 500 ms, acrescido do overhead de temporização e impressão. Entre as versões concorrentes, a V2 (média de 234.03 ms) foi mais lenta que a V3 (202.75 ms). A V1 não concluiu devido ao deadlock.

8. Qual solução apresentou maior tempo de espera pelos recursos?

A V2 apresentou o maior tempo de espera pelos recursos: espera média de 4.88 ms por ciclo, cerca de 52% superior à da V3 (3.21 ms), e o maior pico de espera (23.84 ms, contra 12.47 ms da V3). Isso decorre do gargalo no árbitro, onde filósofos aguardam mesmo quando seus garfos estão livres. A V2 também foi a mais instável: sua espera média variou de 3.87 ms a 7.10 ms entre execuções, enquanto a da V3 ficou entre 3.00 ms e 3.47 ms. Na V1, a espera foi indefinida, pois as goroutines ficaram bloqueadas permanentemente.

9. Quantos filósofos conseguiram comer simultaneamente em cada solução?

Na V0, apenas 1 filósofo comia por vez. Na V1, nenhum filósofo chegou a comer (0), pois todos ficaram bloqueados antes da primeira refeição. Nas versões V2 e V3, até 2 filósofos comeram simultaneamente em todas as 5 execuções, que é o máximo possível para 5 filósofos e 5 garfos, já que cada refeição exige dois garfos adjacentes e 5 garfos só formam 2 pares disjuntos.

10. Qual foi o impacto da sincronização excessiva da V2?

O árbitro centralizado atuou como ponto único de contenção: enquanto um filósofo aguarda um garfo ocupado por um vizinho que está comendo, todos os demais que querem comer ficam bloqueados no árbitro, mesmo que seus próprios garfos estejam livres. Em média, isso elevou o tempo total para 234.03 ms, 31.28 ms (15.4%) a mais que a V3, e a espera média para 4.88 ms por ciclo. A sincronização excessiva também tornou o desempenho imprevisível: o desvio padrão do tempo total da V2 foi de 28.96 ms (de 211.60 ms a 277.08 ms), contra 9.07 ms na V3, pois basta um filósofo demorar no árbitro para atrasar todos os outros.

11. A solução com maior sincronização foi necessariamente a de melhor desempenho?

Não. A V2 possui o maior grau de sincronização (Mutex árbitro global + Mutex por garfo), mas foi a versão concorrente com pior desempenho em todas as métricas: tempo total médio de 234.03 ms contra 202.75 ms da V3, espera média de 4.88 ms contra 3.21 ms e maior espera de 23.84 ms contra 12.47 ms. Mais sincronização garantiu a corretude (ausência de deadlock), mas restringiu a concorrência; a V3 obteve a mesma garantia com menos sincronização, apenas ordenando a aquisição dos garfos.

12. Qual é a relação entre o grau de concorrência e o tempo total de execução?

Quanto maior o grau de concorrência, menor o tempo total. A V0, com 1 filósofo ativo por vez, levou em média 656.94 ms; a V2 e a V3, com até 2 filósofos comendo simultaneamente, levaram 234.03 ms e 202.75 ms, um speedup de aproximadamente 2.8x e 3.2x, respectivamente. O speedup é maior

que 2 (número máximo de filósofos comendo juntos) porque, nas versões concorrentes, os filósofos também pensam em paralelo, enquanto na V0 até o tempo de pensar é somado sequencialmente. A relação, porém, não é estritamente proporcional: V2 e V3 tiveram o mesmo máximo de filósofos comendo (2), mas a V3 foi 13.4% mais rápida, porque filósofos não vizinhos adquirem seus garfos em paralelo, sem passar por um ponto único de serialização.